

Worksharing

Worksharing

- Las construcciones worksharing permiten definir el trabajo a realizar por las tareas implícitas:
 - Worksharing - loop (**for**): explota el **paralelismo de datos** mediante la asignación de iteraciones de bucles.
 - Worksharing **single** y **sections**: permiten **paralelismo de tareas** (funcional).
- Las construcciones worksharing:
 - No tienen un **barrier** al inicio de la región.
 - Tienen un **barrier** implícito al final de la región, a menos que se especifique la cláusula **nowait**.
 - No pueden contener otro worksharing sin una construcción **parallel** entre medio.

Worksharing-loop

Construcción for

Distribuye las iteraciones del bucle entre los threads que encuentran la construcción.

```
#pragma omp for [ cláusulas ]  
    for (init-expr; test-expr; inc-expr)
```

- **init-expr** debe ser de la forma:
 índice = **x** siendo **x** invariante

(es decir, no debe variar durante la ejecución del bucle)

- **test-expr** debe ser de la forma: **índice** **op** **x** (o invertido), siendo **x** invariante. La operación **op** puede ser: **<**, **<=**, **>**, **>=**
- **inc-expr** debe ser tal que el incremento del bucle sea invariante, y ser aditivo (**+**) o sustractivo (**-**)

Las cláusulas pueden ser:

- private**, **firstprivate**
 (lista de variables)
- reduction** (operador : lista de variables)
- schedule** (schedule-kind)
- nowait**
- collapse(n)**

Consideraciones

- Las iteraciones no deben tener dependencias de datos.
- Los tipos de datos válidos para variables índices son **enteros** (cualquier tamaño) y **punteros**.
 - Se definen automáticamente como **private**.
- El atributo de compartición de datos por defecto es **shared**.
- La cláusula **nowait** elimina la barrera implícita al final del bucle.
- Puede usarse de manera combinada con la construcción **parallel**:

```
#pragma omp parallel for
```

Ejemplo

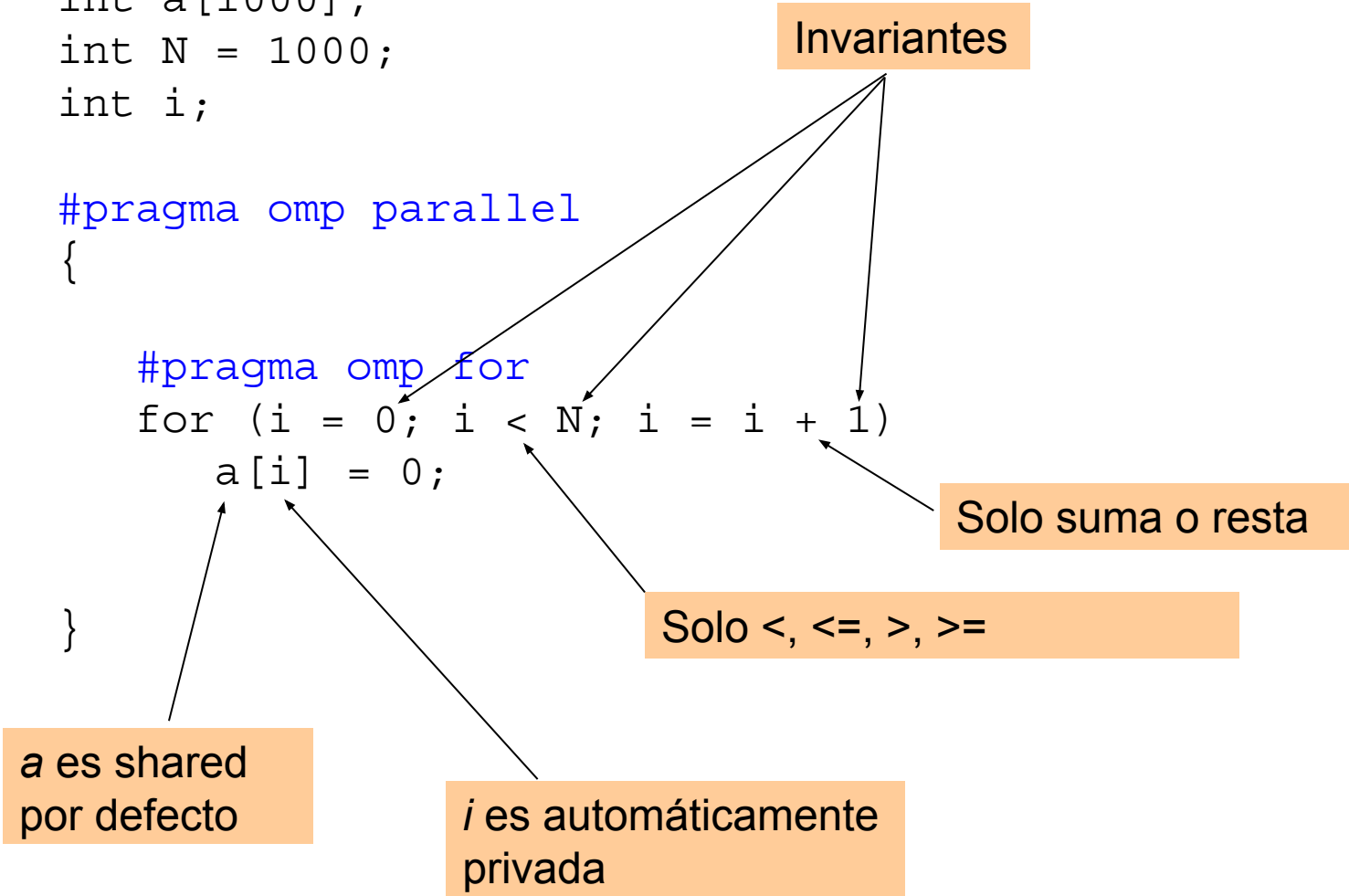
```
int a[1000];  
int N = 1000;  
int i;
```

```
#pragma omp parallel  
{
```

```
  #pragma omp for  
  for (i = 0; i < N; i = i + 1)  
    a[i] = 0;
```

```
}
```

Invariantes



Solo suma o resta

Solo <, <=, >, >=

a es shared
por defecto

i es automáticamente
privada

Ejemplo

```
int m[1000][500];  
int N = 1000, M = 500;  
int i, j;
```

```
#pragma omp parallel
```

```
{
```

```
    #pragma omp for private (j)
```

```
    for (i = 0; i < N; i++)
```

```
        for (j = 0; j < M; j++)
```

```
            m[i][j] = 0;
```

```
}
```

}

Cómputo a realizar por
cada iteración de *i*

j debe declararse explícitamente
como privada

Ejemplo: uso conjunto con parallel

Programas equivalentes

```
int a[1000];
int N = 1000;
int i;

#pragma omp parallel
{
    #pragma omp for
    for (i = 0; i < N; i = i + 1)
        a[i] = 0;
}
```

```
int a[1000];
int N = 1000;
int i;

#pragma omp parallel for
for (i = 0; i < N; i = i + 1)
    a[i] = 0;
```

Esta conjunción sólo es posible porque la región paralela solo contiene a la construcción `for`.

Ejemplo: cláusula nowait

```
#pragma omp parallel
{
    #pragma omp for nowait
    for (i = 0; i < n; i++)
        v[i] = 0;

    #pragma omp for
    for (i = 0; i < n; i++)
        a[i] = 0;
}
```

Ambos bucles se pueden solapar porque son independientes

```
#pragma omp parallel
{
    #pragma omp for nowait
    for (i = 0; i < n; i++)
        v[i] = 0;

    #pragma omp for
    for (i = 0; i < n; i++)
        a[i] = v[i] * v[i];
}
```

Los bucles son dependientes, por lo tanto hay condición de carrera

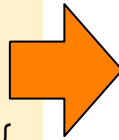
Dependencias entre bucles

- El “renombrado de variables” puede lograrse utilizando la cláusula **private**.

Hay condición de carrera

```
int a[1000], b[1000];
int N = 1000;
int i, temp;

#pragma omp parallel for
  for (i = 0; i < N; i = i + 1) {
    temp = a[i];
    a[i] = b[i];
    b[i] = temp;
  }
```



No hay condición de carrera

```
int a[1000], b[1000];
int N = 1000;
int i, temp;

#pragma omp parallel for private(temp)
  for (i = 0; i < N; i = i + 1) {
    temp = a[i];
    a[i] = b[i];
    b[i] = temp;
  }
```

$$\begin{aligned} I_1 \cap O_2 &= \emptyset \\ I_2 \cap O_1 &= \emptyset \\ O_1 \cap O_2 &= \emptyset \end{aligned}$$

- Las dependencias verdaderas no pueden evitarse, debe cambiarse el algoritmo.

cláusula reduction

La cláusula **reduction** de la construcción **for** funciona igual que en la construcción **parallel**.

```
int a = 0;
#pragma omp parallel
{
    int i;
    #pragma omp for reduction (+:a)
    for (i = 0; i < 1000; i++) {
        a = a + 1;
    }
    printf("Valor de a: %d\n", a);
}
```

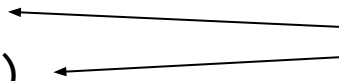
Se crean copias privadas de *a*

La variable *a* se actualiza a partir de los valores parciales (acumulados por cada thread) y el valor original de *a*

cláusula collapse(n)

- Permite distribuir el trabajo de un conjunto de n bucles anidados.
- Las iteraciones de bucles anidados se colapsan en un único espacio de iteraciones de gran tamaño, ordenadas según el orden natural de ejecución secuencial.
 - Cada bucle participante debe cumplir con las condiciones ya establecidas para `init-expr`, `test-expr`, `inc-expr`

```
#pragma omp for collapse (2)
for (i = 0; i < N; i++)
    for (j = 0; j < M; j++)
        hola(i,j);
```



Los bucles i y j se pliegan y las iteraciones se distribuyen entre todos los threads. Las variables i y j son automáticamente privadas.

Worksharing-loop
Cláusula schedule

schedule

- Determina cuáles iteraciones son ejecutadas por cada thread del grupo:

`schedule(opción)`

La cláusula `schedule` va seguida de una de las siguientes opciones:

- Si no se especifica, se utiliza el **schedule por defecto** definido por la implementación particular de OpenMP.

`static`
`dynamic`
`guided`

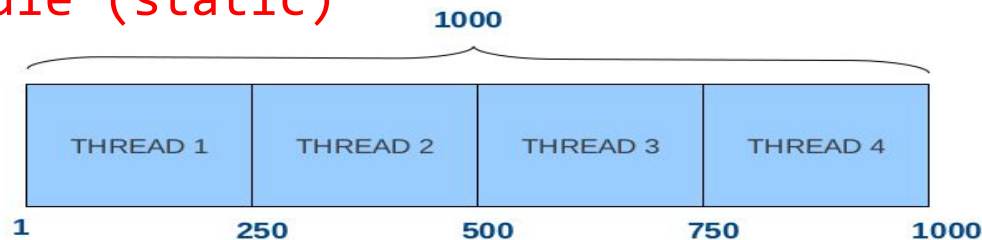
- Las distintas planificaciones asignan trabajos en unidades llamadas **piezas**:
 - **pieza**: subconjunto de iteraciones contiguas.
 - **tamaño_pieza**: indica el tamaño de la pieza en número de iteraciones.

schedule(static[, N])

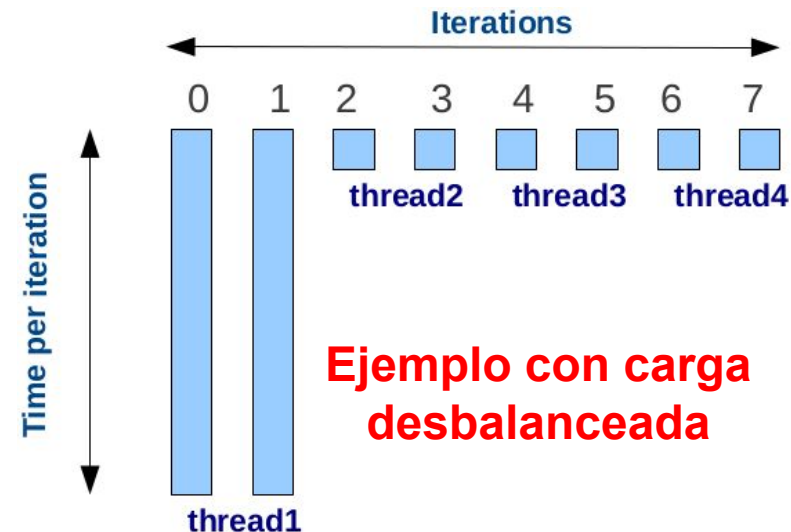
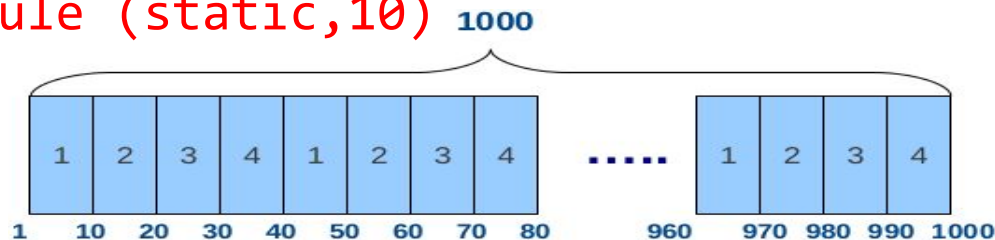
La asignación de las iteraciones a threads se hace **antes de iniciar la ejecución del bucle y se mantiene hasta la finalización del mismo**:

- El espacio de iteraciones se divide en **piezas de tamaño N**, y se asignan a los threads en forma *round-robin* ordenados por número de thread.
- Si no se especifica **N**, las piezas serán (aproximadamente) de igual tamaño, y se asignará como máximo una pieza por thread.
- La planificación estática tiene **baja sobrecarga de gestión del paralelismo**, pero el reparto de iteraciones en partes iguales puede producir **desbalance de carga**.

schedule (static)



schedule (static,10)



Planificaciones dinámicas

La asignación de las iteraciones a threads se va realizando a medida que avanza la ejecución del bucle.

Las planificaciones dinámicas tienen **alta sobrecarga de gestión del paralelismo**, pero al tener un reparto de iteraciones por demanda producen normalmente un **buen balance de carga**.

Hay dos planificaciones dinámicas: **dynamic** y **guided**

Ambas planificaciones permiten regular el tamaño de la pieza, el cual impacta en la sobrecarga y el balance de carga.

`schedule(dynamic[, N])`

El espacio de iteraciones se divide en piezas de tamaño **N**. Cada thread recibe una pieza y cuando termina el trabajo recibe otra pieza.

- Si no se especifica el tamaño de las piezas, **N = 1**

schedule(guided[, N])

Similar a **dynamic** pero el tamaño de las piezas decrece a medida que se avanza en el procesamiento de las iteraciones, hasta llegar a piezas de tamaño **N**.

- Si no se especifica el tamaño de las piezas, **N = 1**

El tamaño de una pieza se calcula aproximadamente de la siguiente manera:

```
Pieza = iteraciones_restantes / número_de_threads;  
si (Pieza < N) {  
    Pieza = N;  
    si (Pieza > iteraciones_restantes)  
        Pieza = iteraciones_restantes;  
}
```